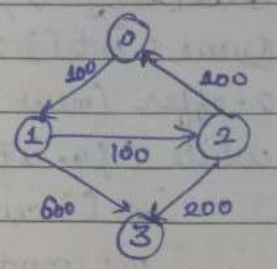
NAME → CHANDAN SINGLA
UID → 23BCS12759
BATCH → KRG-1A

Date 30/3/26
Page

COMPETITIVE CODING - II

Exp-7

Testcase 1:- Input: n = 4 [1,2,100]
flights :- [[0,1,100], [2,0,100], [1,3,600], [2,3,200]]
src = 0, dest = 3, k = 1
Output :- 700




Algorithm :→

Step1:- Make a graph (i.e. Adjacency list)

Step2:- Make a distance array where dist[i] = minimum cost to reach node i. & initialize it with ∞.
dist[src] = 0.

Step3:- Initialize a Queue (src, cost = 0, stops = 0).

Step4:- Start BFS traversal
↳ Pop element from queue.
↳ If stops > k → skip.
↳ for each neighbour :
↳ Calculate new cost.
↳ if new cost < dist[neighbour]
↳ Update dist
↳ Push into queue with stops++;

Step5:- ↳ Return Result
↳ If destination not reached → return -1.
↳ Else → return dist[dest].

# Pseudo code :-

vivo Y16
Mar 31, 2026, 15:38

```
function cheapestFlight( int n, int [][] flights, int src,
    int dest, int k){
  int[] dist = new int[n];
  Arrays.fill( dist, Integer.MAX_VALUE);
  dist[src] = 0;
  Queue<int[]> q = new LinkedList<>();
  q.offer (new int[]{ src, 0, 0});
  while (!q.isEmpty()){
      int[] curr = q.poll();
      int node = curr[0];
      int cost = curr[1];
      int stops = curr[2];
      if( stops > k) continue;
      for( int[] neighbor : adj.get(node)){
          int newCost = cost + neighbor[1];
          int nbr = neighbor[0];
          if (newCost < dist[nbr]){
              dist[nbr] = newCost;
              q.offer( new int[]{ nbr, newCost, stops+1});
          }
      }
  }
  if (dist[dest] == Integer.MAX_VALUE){ return -1; }
  else
      return dist[dest];
}
```

Time-Complexity = O(E+V)

vivo Y16
Mar 31, 2026, 15:38